

Chương 2 Tầng ứng dụng

A note on the use of these ppt slides:

We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a lot of work on our part. In return for use, we only ask the following:

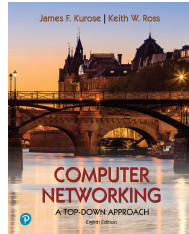
- ❖ If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- ❖ If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

Thanks and enjoy! JFK/KWR

© All material copyright 1996-2012
© J.F. Kurose and K.W. Ross, All Rights Reserved

Người dịch: Nguyễn Thị Thanh Thủy

Tài liệu được dịch cho mục đích giảng dạy



*Computer Networking:
A Top-Down Approach*
8th edition
Jim Kurose, Keith Ross
Pearson, 2020

Tầng ứng dụng 2-1

Chương 2: Tầng ứng dụng

Mục tiêu:

- ❖ Khái niệm, các vấn đề triển khai giao thức ứng dụng mạng
 - Các mô hình dịch vụ tầng giao vận
 - Mô hình khách-chủ (client-server)
 - Mô hình điểm-điểm (peer-to-peer)
- ❖ Nghiên cứu một số giao thức tầng ứng dụng
 - HTTP
 - FTP
 - SMTP / POP3 / IMAP
 - DNS
 - Các hệ thống video streaming, CDNs
- ❖ Tạo một ứng dụng mạng
 - socket API

Tầng ứng dụng 2-2

Chương 2: Nội dung

2.1. Nguyên lý của ứng dụng mạng

- 2.1.1. Các kiến trúc của ứng dụng mạng
- 2.1.2. Truyền thông giữa các tiến trình
- 2.1.3. Các dịch vụ giao vận

2.2. Web và HTTP

2.3. FTP

2.4. Thư điện tử

2.5. DNS (Domain Name Systems)

2.6. Ứng dụng Peer-to-peer

2.7. Video streaming và các mạng phân phối nội dung

2.8. Lập trình socket với UDP và TCP

Tầng ứng dụng 2-3

Một số ứng dụng mạng

- ❖ Mạng xã hội
- ❖ Web
- ❖ Tin nhắn văn bản (text messaging)
- ❖ Thư điện tử (e-mail)
- ❖ Trò chơi nhiều người trên mạng
- ❖ streaming stored video (YouTube, Hulu, Netflix)
- ❖ Chia sẻ file P2P
- ❖ Điện thoại Internet (voice over IP, ví dụ Skype)
- ❖ Hội thảo video thời gian thực
- ❖ Các ứng dụng tìm kiếm
- ❖ Truy nhập từ xa (remote login)
- ❖ ...

Tầng ứng dụng 2-4

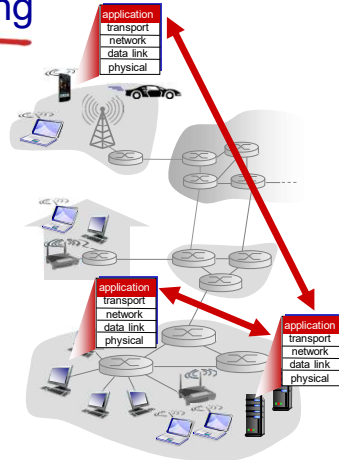
Tạo một ứng dụng mạng

Viết chương trình để:

- ❖ Chạy trên các *hệ thống đầu cuối* (khác nhau)
- ❖ Truyền thông qua mạng
- ❖ Ví dụ: phần mềm máy chủ web (web server) truyền thông với phần mềm trình duyệt (browser software)

Không cần viết chương trình ứng dụng cho các thiết bị trong phần lõi của mạng

- ❖ Các thiết bị trong phần lõi của mạng không chạy các ứng dụng người dùng.
- ❖ Các ứng dụng chạy trên thiết bị đầu cuối cho phép phát triển và phổ biến ứng dụng một cách nhanh chóng.



Tầng ứng dụng 2-5

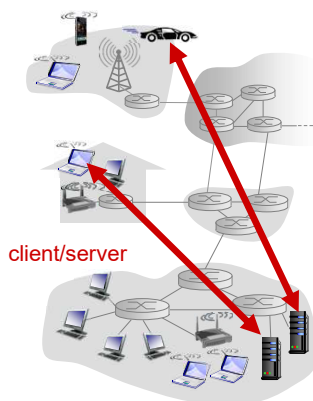
Kiến trúc của ứng dụng

Các ứng dụng có thể có kiến trúc dạng:

- ❖ Client-server (khách-chủ)
- ❖ Peer-to-peer (P2P, ngang hàng)

Tầng ứng dụng 2-6

Kiến trúc client-server



server:

- ❖ Là host luôn hoạt động
- ❖ Có địa chỉ IP cố định
- ❖ Thường có trong các trung tâm dữ liệu, để mở rộng quy mô.

client:

- ❖ Liên lạc, truyền thông với server
- ❖ Có thể được kết nối liên tục vào mạng hoặc không
- ❖ Có thể có địa chỉ IP thay đổi
- ❖ Không truyền thông trực tiếp với client khác

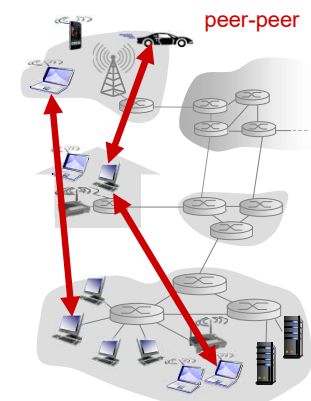
Ví dụ:

- ❖ HTTP, IMAP, FTP

Tầng ứng dụng 2-7

Kiến trúc P2P

- ❖ Không có server luôn hoạt động
- ❖ Các hệ thống đầu cuối (peer) truyền thông trực tiếp với nhau
- ❖ Mỗi peer yêu cầu dịch vụ từ một peer nào đó, và cung cấp dịch vụ lại cho các peer khác.
 - Có khả năng tự mở rộng – peer mới mang lại khả năng dịch vụ mới, cũng như có những yêu cầu dịch vụ mới.
- ❖ Các peer không kết nối liên tục và có thể thay đổi địa chỉ IP
 - Quản lý phức tạp
- ❖ Ví dụ: chia sẻ file P2P [BitTorrent]



Tầng ứng dụng 2-8

Tiến trình truyền thông

Tiến trình: chương trình chạy trên một host

- Trên cùng một host, hai tiến trình truyền thông với nhau qua **truyền thông tiến trình nội bộ** (được xác định bởi hệ điều hành)
- Các tiến trình trên các host khác nhau truyền thông với nhau bằng cách trao đổi các **thông điệp**

client, server

Tiến trình client: tiến trình khởi tạo truyền thông

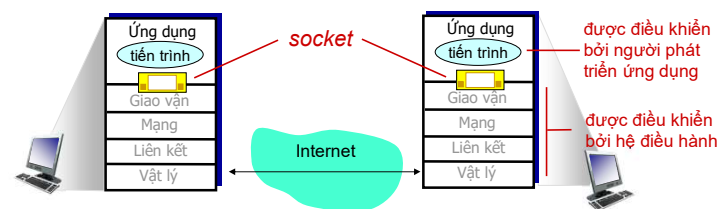
Tiến trình server: tiến trình chờ được tiếp xúc

- Lưu ý:** ứng dụng theo kiến trúc P2P có cả tiến trình client & tiến trình server

Tầng ứng dụng 2-9

Socket

- Các tiến trình gửi/nhận các thông điệp đến/từ **socket** của nó
- Socket tương tự như một cửa vào/ra
 - Tiến trình gửi đẩy thông điệp ra bên ngoài cửa
 - Tiến trình gửi dựa trên cơ sở hạ tầng tầng giao vận ở phía bên kia cửa để truyền thông điệp đến socket của tiến trình nhận.
 - Hai socket phụ thuộc vào nhau: mỗi socket ở một phía.



Tầng ứng dụng 2-10

Định địa chỉ cho tiến trình

- Để nhận các thông điệp, tiến trình phải có **định danh (identifier)**
- Thiết bị host phải có địa chỉ IP 32-bit duy nhất
- Câu hỏi:** Địa chỉ IP của host mà trên đó tiến trình chạy có đủ để xác định định danh của tiến trình không?
 - Trả lời:** Không, do nhiều tiến trình có thể cùng chạy trên một host.
- Định danh** bao gồm cả **địa chỉ IP** và **số hiệu cổng** được kết hợp với tiến trình trên host.
- Ví dụ các số hiệu cổng:
 - HTTP server: 80
 - mail server: 25
- Để gửi thông điệp HTTP tới web server gaia.cs.umass.edu:
 - Địa chỉ IP:** 128.119.245.12
 - Số hiệu cổng:** 80

Tầng ứng dụng 2-11

Các giao thức tầng ứng dụng

- Các loại thông điệp được trao đổi**
 - Ví dụ: thông điệp yêu cầu (request), thông điệp đáp ứng (response)
- Cú pháp của thông điệp:**
 - Các trường trong thông điệp & mô tả
- Ngữ nghĩa của thông điệp**
 - Ý nghĩa thông tin của các trường
- Quy tắc** về thời gian và cách thức các tiến trình gửi và đáp ứng thông điệp
- Các giao thức mở (công khai):**
 - Được định nghĩa trong RFCs (mọi người đều có thể truy nhập vào).
 - Cho phép tương tác
 - Ví dụ: HTTP, SMTP
- Các giao thức riêng (độc quyền):**
 - Ví dụ: Skype, Zoom

Tầng ứng dụng 2-12

Một ứng dụng cần những dịch vụ giao vận nào?

Toàn vẹn dữ liệu

- ❖ Một số ứng dụng (ví dụ: truyền file, web) yêu cầu truyền tin cậy 100%
- ❖ Các ứng dụng khác (ví dụ: audio) có thể chịu một số mất mát.

Thời gian thực

- ❖ Một số ứng dụng (ví dụ: điện thoại Internet, trò chơi tương tác) yêu cầu độ trễ thấp mới “hiệu quả”

Băng thông

- ❖ Một số ứng dụng (ví dụ: đa phương tiện) yêu cầu băng thông tối thiểu để đảm bảo “hiệu quả”.
- ❖ Các ứng dụng khác (“các ứng dụng mềm dẻo”) có thể dùng băng thông nào cũng được.

An toàn bảo mật

- ❖ Mã hóa, đảm bảo toàn vẹn dữ liệu, ...

Tăng ứng dụng 2-13

Một số yêu cầu dịch vụ giao vận với các ứng dụng phổ biến

Ứng dụng	Mất mát dữ liệu	Thông lượng	Thời gian thực
Truyền file	không	mềm dẻo	không
Thư điện tử	không	mềm dẻo	không
Web	không	mềm dẻo	không
Audio/video thời gian thực	chịu lỗi	audio: 5kbps-1Mbps video: 10kbps-5Mbps	có, 100 ms
Lưu trữ audio/video	chịu lỗi	như trên	có, vài giây
Trò chơi tương tác	chịu lỗi	một vài kbps	có, 10 ms
Tin nhắn nhanh	không	mềm dẻo	có và không

Tăng ứng dụng 2-14

Các dịch vụ giao thức giao vận Internet

Dịch vụ TCP:

- ❖ **Truyền tin cậy** giữa tiến trình gửi và tiến trình nhận
- ❖ **Điều khiển luồng**: bên gửi không lấn át bên nhận
- ❖ **Điều khiển tắc nghẽn**: điều tiết bên gửi khi mạng bị quá tải
- ❖ **Không cung cấp**: thời gian thực, đảm bảo băng thông tối thiểu, an toàn bảo mật
- ❖ **Hướng kết nối**: yêu cầu thiết lập kết nối giữa tiến trình client và tiến trình server

Dịch vụ UDP:

- ❖ **Truyền dữ liệu không tin cậy** giữa tiến trình gửi và tiến trình nhận
- ❖ **Không cung cấp**: truyền tin cậy, điều khiển luồng, điều khiển tắc nghẽn, thời gian thực, đảm bảo băng thông, an toàn bảo mật, hoặc thiết lập kết nối

Câu hỏi: Tại sao lại dùng cả hai dịch vụ? Dùng UDP để làm gì?

Tăng ứng dụng 2-15

Các ứng dụng Internet: các giao thức tầng ứng dụng và giao vận

Ứng dụng	Giao thức tầng ứng dụng	Giao thức tầng giao vận
Truyền file	FTP [RFC 959]	TCP
Thư điện tử	SMTP [RFC 5321]	TCP
Web	HTTP [RFC 7230, 9110]	TCP
Điện thoại Internet	SIP [RFC 3261], RTP [RFC 3550], hoặc giao thức độc quyền	TCP hoặc UDP
streaming audio/video	HTTP [RFC 7230], DASH	TCP
Trò chơi tương tác	WOW, FPS (giao thức độc quyền)	UDP hoặc TCP

Tăng ứng dụng 2-16

Bảo mật TCP

TCP & UDP

- ❖ Không mã hóa
- ❖ Mật khẩu dạng bản rõ được gửi vào socket để truyền trên Internet theo dạng bản rõ

Bảo mật tầng giao vận (Transport Layer Security, TLS)

- ❖ Cung cấp kết nối TCP được mã hóa
- ❖ Toàn vẹn dữ liệu
- ❖ Xác thực đầu cuối

TLS được triển khai tại tầng ứng dụng

- ❖ Các ứng dụng sử dụng thư viện SSL, tiếp đó sử dụng TCP
- ❖ Mật khẩu dạng bản rõ được gửi vào socket để truyền trên Internet theo dạng đã được mã hóa.

Chương 2: Nội dung

2.1. Nguyên lý của ứng dụng mạng

- 2.1.1. Các kiến trúc của ứng dụng mạng
- 2.1.2. Truyền thông giữa các tiến trình
- 2.1.3. Các dịch vụ giao vận

2.2. Web và HTTP

2.3. FTP

2.4. Thư điện tử

2.5. DNS (Domain Name Systems)

2.6. Ứng dụng Peer-to-peer

2.7. Video streaming và các mạng phân phối nội dung

2.8. Lập trình socket với UDP và TCP

Tăng ứng dụng 2-18

Web và HTTP

Một số thuật ngữ:

- ❖ **Trang web** bao gồm một số **đối tượng (object)**
- ❖ Đối tượng có thể là tệp HTML, ảnh JPEG, Java applet, tệp audio,...
- ❖ Trang web bao gồm **tệp HTML cơ bản** chứa **một số đối tượng được tham chiếu**
- ❖ Mỗi đối tượng được định địa chỉ bởi một **URL**, ví dụ:

`www.someschool.edu/someDept/pic.gif`

Tên host

Tên đường dẫn

Tăng ứng dụng 2-19

Khái quát về HTTP

HTTP: hypertext transfer protocol

- ❖ Giao thức tầng ứng dụng của Web
- ❖ Mô hình client/server
 - **client**: trình duyệt (browser) yêu cầu, nhận (sử dụng giao thức HTTP), và “hiển thị” các đối tượng Web
 - **server**: Máy chủ web (web server) gửi (sử dụng giao thức HTTP) các đối tượng đáp ứng cho yêu cầu.



Tăng ứng dụng 2-20

Khái quát về HTTP

HTTP dùng TCP:

- ❖ Client khởi tạo kết nối TCP (tạo socket) tới server, cổng 80
- ❖ Server chấp nhận kết nối TCP từ client
- ❖ Thông điệp HTTP (thông điệp giao thức tầng ứng dụng) được trao đổi giữa trình duyệt (HTTP client) và máy chủ Web (HTTP server)
- ❖ Đóng kết nối TCP

HTTP là “không trạng thái”

- ❖ Server không lưu giữ thông tin về những yêu cầu trước đó của client.

Vấn đề liên quan

Giao thức lưu giữ “trạng thái” khá phức tạp!

- ❖ Lịch sử quá khứ (trạng thái) cần phải được lưu giữ
- ❖ Nếu server/client bị sự cố, thì quan điểm về “trạng thái” của chúng có thể không nhất quán, cần phải được điều chỉnh.

Tăng ứng dụng 2-21

Kết nối HTTP: 2 loại

HTTP không bền vững

- ❖ Mở kết nối TCP
 - ❖ Chỉ có tối đa một đối tượng được gửi qua một kết nối TCP
 - ❖ Kết nối TCP sau đó sẽ được đóng lại.
- Việc tải về nhiều đối tượng sẽ yêu cầu nhiều kết nối.

HTTP bền vững

- ❖ Kết nối TCP được mở tới một server
- ❖ Nhiều đối tượng có thể được gửi qua một kết nối TCP duy nhất giữa client và server đó
- ❖ Kết nối TCP sau đó sẽ được đóng lại.

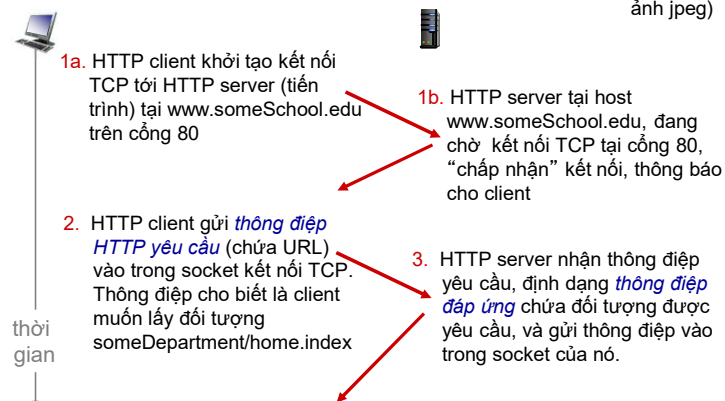
Tăng ứng dụng 2-22

Ví dụ: HTTP không bền vững

Giả sử người dùng gõ URL:

`www.someSchool.edu/someDepartment/home.index`

(chứa văn bản, tham chiếu tới 10 ảnh jpeg)



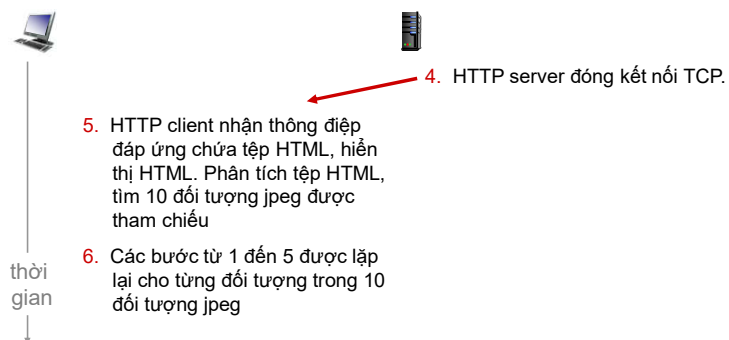
Tăng ứng dụng 2-23

Ví dụ: HTTP không bền vững

Giả sử người dùng gõ URL:

`www.someSchool.edu/someDepartment/home.index`

(chứa văn bản, tham chiếu tới 10 ảnh jpeg)



Tăng ứng dụng 2-24

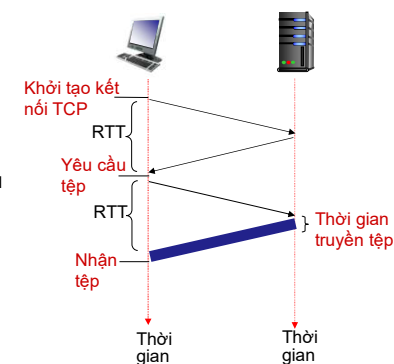
HTTP không bền vững: thời gian đáp ứng

RTT (định nghĩa): thời gian để một gói tin nhỏ đi từ client đến server và quay lại.

Thời gian đáp ứng HTTP:

- ❖ Một RTT để khởi tạo kết nối TCP
- ❖ Một RTT để gửi HTTP yêu cầu và một vài byte đầu tiên của HTTP đáp ứng được trả về
- ❖ Thời gian truyền tệp/đối tượng

Thời gian đáp ứng của HTTP không bền vững = $2RTT + \text{thời gian truyền tệp}$



Tăng ứng dụng 2-25

HTTP bền vững

Vấn đề với HTTP không bền vững:

- ❖ Cần 2 RTT cho mỗi đối tượng
- ❖ Hệ điều hành liên quan đến *mỗi* kết nối TCP
- ❖ Các trình duyệt thường mở nhiều kết nối TCP song song để lấy các đối tượng được tham chiếu

HTTP bền vững (HTTP 1.1):

- ❖ Server để mở kết nối sau khi gửi đáp ứng
- ❖ Chuỗi các thông điệp HTTP tiếp theo giữa cùng client/server sẽ được gửi thông qua kết nối mở này
- ❖ Client gửi yêu cầu ngay sau khi gặp một đối tượng được tham chiếu
- ❖ Ít nhất là một RTT cho tất cả các đối tượng được tham chiếu (giảm thời gian phản hồi xuống một nửa).

Tăng ứng dụng 2-26

Thông điệp HTTP yêu cầu

- ❖ Có hai loại thông điệp HTTP: *yêu cầu* và *đáp ứng*
- ❖ **Thông điệp HTTP yêu cầu:**
 - ASCII (định dạng con người có thể đọc được)

Dòng yêu cầu (các lệnh GET, POST, HEAD)

Các dòng tiêu đề (header)

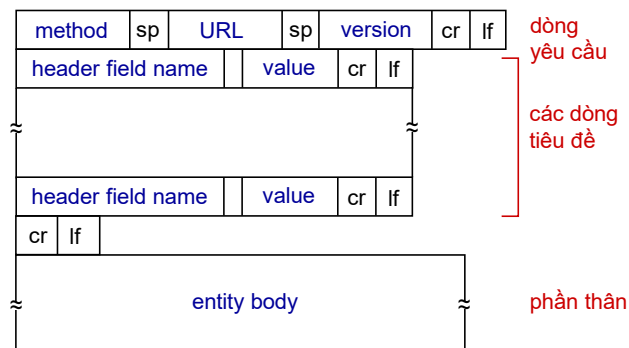
Ký tự trở về đầu dòng, xuống dòng ở đầu một dòng sẽ cho biết điểm cuối của dòng tiêu đề

```
GET /index.html HTTP/1.1\r\nHost: www-net.cs.umass.edu\r\nUser-Agent: Firefox/3.6.10\r\nAccept: text/html,application/xhtml+xml\r\nAccept-Language: en-us,en;q=0.5\r\nAccept-Encoding: gzip,deflate\r\nAccept-Charset: ISO-8859-1,utf-8;q=0.7\r\nKeep-Alive: 115\r\nConnection: keep-alive\r\n\r\n
```

ký tự trở về đầu dòng
ký tự xuống dòng

Tăng ứng dụng 2-27

Thông điệp HTTP yêu cầu: định dạng tổng quát



Tăng ứng dụng 2-28

Các thông điệp HTTP yêu cầu khác

Phương thức POST:

- Trang web thường chứa form input
- Đầu vào (input) được tải lên server trong phần thân thực thể (entity body) của thông điệp yêu cầu HTTP POST.

Phương thức GET (để gửi dữ liệu đến server):

- Bao gồm dữ liệu của người dùng trong trường URL của thông điệp yêu cầu HTTP GET (theo sau dấu '?'):

www.somesite.com/animalsearch?monkeys&banana

Phương thức HEAD:

- Các yêu cầu về tiêu đề (chỉ) được trả về khi có URL xác định được yêu cầu bằng một phương thức HTTP GET.

Phương thức PUT:

- Tải file (/đối tượng) mới lên server
- Thay thế hoàn toàn file tồn tại tại URL xác định bằng nội dung bên trong phần thân thực thể của thông điệp yêu cầu HTTP POST.

Tăng ứng dụng 2-29

Thông điệp HTTP đáp ứng

Dòng trạng thái
(Giao thức
mã trạng thái,
cụm từ trạng
thái)

Các dòng
tiêu đề

Dữ liệu,
ví dụ
tệp HTML
yêu cầu

```
HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02
GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-
1\r\n
\r\n
data data data data data ...
```

Tăng ứng dụng 2-30

Các mã trạng thái HTTP đáp ứng

- ❖ Mã trạng thái xuất hiện ngay trong dòng đầu tiên của thông điệp đáp ứng từ server đến client.
- ❖ Ví dụ một số mã:

200 OK

- Yêu cầu thành công, đối tượng yêu cầu nằm ở phía sau thông điệp này

301 Moved Permanently

- Đối tượng yêu cầu đã bị di chuyển, vị trí mới được xác định ở phía sau thông điệp này (Location: field)

400 Bad Request

- Server không hiểu thông điệp yêu cầu

404 Not Found

- Tài liệu yêu cầu không có trong server này

505 HTTP Version Not Supported

Tăng ứng dụng 2-31

Tự kiểm tra HTTP (phía client)

1. Telnet đến Web server ưa thích của bạn:

```
telnet cis.poly.edu 80
```

Mở kết nối TCP ở cổng 80 (cổng mặc định của HTTP server) tại cis.poly.edu. Nhập yêu cầu gì đó và gửi tới cổng 80 tại cis.poly.edu

2. Nhập yêu cầu trong lệnh GET HTTP:

```
GET /kurose_ross/interactive/index.php
HTTP/1.1
```

```
Host: cis.poly.edu
```

Bảng cách gõ lệnh này (nhấn enter 2 lần), nghĩa là gửi yêu cầu GET tối thiểu (nhưng đầy đủ) tới HTTP server

3. Xem thông điệp đáp ứng được gửi về từ HTTP server!
(hoặc dùng Wireshark để xem thông điệp HTTP yêu cầu/đáp ứng bất được)

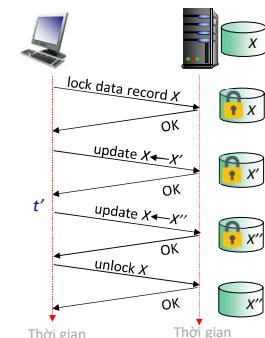
Tăng ứng dụng 2-32

Duy trì trạng thái user-server: cookie

Nhớ lại: HTTP GET/tương tác phản hồi là **không trạng thái**

- ❖ Không có khái niệm về việc trao đổi các thông điệp HTTP nhiều bước để hoàn thành một "giao dịch" trên Web
 - Không cần client/server theo dõi "trạng thái" trao đổi nhiều bước
 - Tất cả các HTTP yêu cầu đều độc lập với nhau
 - Không cần client/server "khôi phục" giao dịch đã thực hiện được một phần nhưng không bao giờ hoàn thành.

Một giao thức có trạng thái: client thực hiện 2 thay đổi với X, hoặc không thực hiện thay đổi nào.



Câu hỏi: Điều gì sẽ xảy ra khi kết nối mạng hoặc máy client bị lỗi tại thời điểm t' ?

Tăng ứng dụng 2-33

Duy trì trạng thái user-server: cookie

Các trang web và trình duyệt của khách hàng sử dụng **cookie** để duy trì một số trạng thái giữa các giao dịch.

Bốn thành phần:

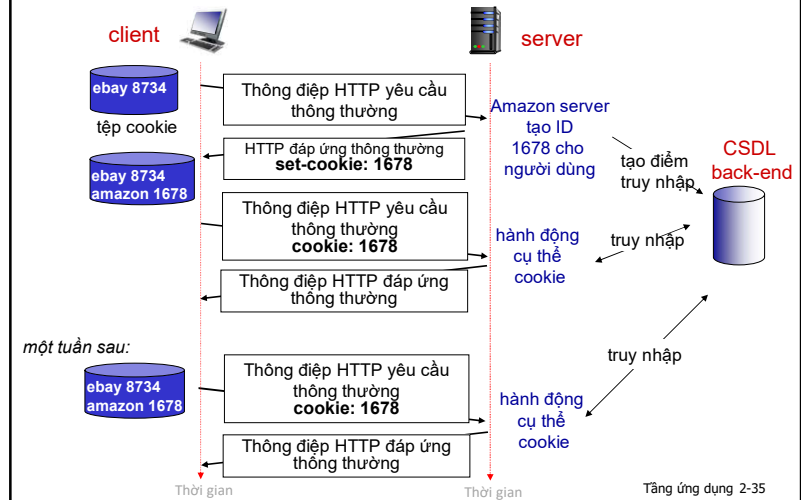
- 1) Dòng tiêu đề cookie của thông điệp HTTP *đáp ứng*
- 2) Dòng tiêu đề cookie trong thông điệp HTTP *yêu cầu* tiếp theo
- 3) Tập cookie được lưu giữ trên host của người dùng, được quản lý bởi trình duyệt của người dùng.
- 4) Cơ sở dữ liệu back-end tại Web site

Ví dụ:

- ❖ Susan sử dụng trình duyệt trên laptop, lần đầu tiên truy nhập vào một trang web thương mại điện tử
- ❖ Khi yêu cầu HTTP khởi tạo đi đến trang web, trang sẽ tạo ra:
 - Một ID duy nhất (còn gọi là "cookie")
 - Điểm truy nhập vào cơ sở dữ liệu back-end cho ID
- ❖ Các HTTP yêu cầu tiếp theo từ Susan tới trang web này sẽ chứa giá trị ID cookie, cho phép trang web "nhận diện" Susan.

Tăng ứng dụng 2-34

Duy trì trạng thái user-server: cookie



HTTP cookie

Cookie có thể được dùng cho những việc gì?

- ❖ Cấp phép
- ❖ Giỏ mua hàng
- ❖ Khuyến nghị
- ❖ Trạng thái phiên làm việc của người dùng (Web, e-mail)

Làm cách nào để lưu "trạng thái":

- ❖ **Tại các điểm cuối giao thức:** duy trì trạng thái tại bên gửi/bên nhận thông qua nhiều giao dịch
- ❖ **Trong các thông điệp:** cookie trong các thông điệp HTTP mang trạng thái.

Tăng ứng dụng 2-36

Ngoài ra

cookie và sự riêng tư:

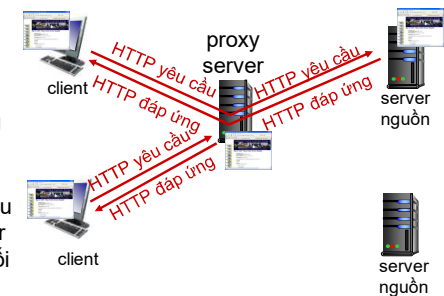
- ❖ Cookie cho phép các site biết nhiều hơn về người dùng
- ❖ Cookie của bên thứ ba (cookie theo dõi) cho phép theo dõi định dạng chung (giá trị cookie) trên nhiều trang web.

Web caches (proxy server)

Mục tiêu: thỏa mãn yêu cầu của client mà không cần liên quan đến server nguồn

- ❖ Người dùng thiết lập trình duyệt: truy nhập Web qua vùng nhớ đệm (cache)
- ❖ Trình duyệt gửi tất cả các yêu cầu HTTP tới cache

- Nếu đối tượng có trong cache: cache sẽ trả về đối tượng
- Ngược lại, cache sẽ yêu cầu đối tượng từ server nguồn, sau đó sẽ trả đối tượng về cho client



Tăng ứng dụng 2-37

Web caches

- ❖ Bộ nhớ cache hoạt động như cả client và server
 - Server đáp ứng cho yêu cầu đầu tiên từ một client
 - Client gửi yêu cầu tới server gốc
 - ❖ Cache thường được cài đặt bởi ISP (trường học, công ty, khu dân cư)
- Tại sao lại cần Web caching?**
- ❖ Giảm thời gian đáp ứng cho yêu cầu của client:
 - Cache gần với client hơn
 - ❖ Giảm lưu lượng truy nhập vào liên kết của tổ chức
 - ❖ Nếu Internet thực hiện cache nhiều:
 - thì sẽ cho phép các nhà cung cấp nội dung "nghèo nàn" phân phối nhiều nội dung đó hơn (tương tự như chia sẻ file P2P).

Tăng ứng dụng 2-38

Ví dụ Caching

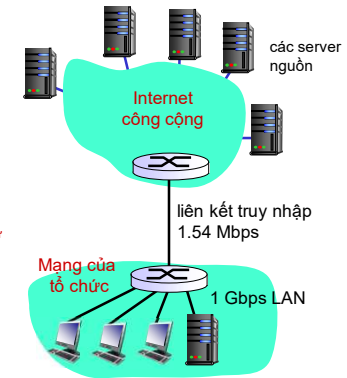
Kịch bản:

- ❖ Tốc độ liên kết truy nhập: 1.54 Mbps
- ❖ RTT từ bộ định tuyến của tổ chức đến server: 2 giây
- ❖ Kích thước đối tượng web: 100K bits
- ❖ Tốc độ yêu cầu trung bình từ trình duyệt đến server gốc: 15/giây
 - tốc độ dữ liệu trung bình tới trình duyệt: 1.50 Mbps

Hiệu suất:

- ❖ Việc sử dụng liên kết truy nhập $\approx .97$
 - ❖ Việc sử dụng LAN: .0015
 - ❖ trễ đầu cuối-đầu cuối
- = trễ Internet +
trễ liên kết truy nhập + trễ LAN
= 2 giây + một số phút + (~1 giây)

Vấn đề: trễ xếp hàng rất lớn khi mức độ sử dụng cao!



Tăng ứng dụng 2-39

Giải pháp 1: mua liên kết truy nhập tốc độ cao hơn

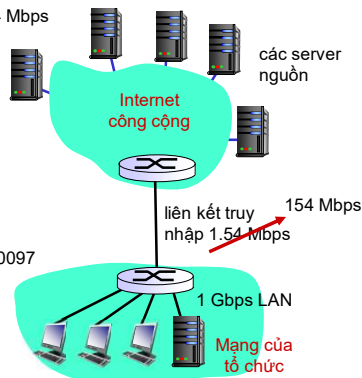
Kịch bản :

- ❖ Tốc độ liên kết truy nhập: 1.54 Mbps $\rightarrow$ 154 Mbps
- ❖ RTT từ bộ định tuyến của tổ chức đến server: 2 giây
- ❖ Kích thước đối tượng web: 100K bits
- ❖ Tốc độ yêu cầu trung bình từ trình duyệt đến server gốc: 15/giây
 - tốc độ dữ liệu trung bình tới trình duyệt: 1.50 Mbps

Hiệu suất:

- ❖ Việc sử dụng liên kết truy nhập $\approx 0.97 \rightarrow 0.0097$
 - ❖ Việc sử dụng LAN: .0015
 - ❖ trễ đầu cuối-đầu cuối
- = trễ Internet +
trễ liên kết truy nhập + trễ LAN
= 2 giây + một số phút + (~1 giây)
msecs

Chi phí: liên kết truy nhập tốc độ cao hơn (đắt đỏ!)



Tăng ứng dụng 2-40

Giải pháp 2: cài đặt web cache

Kịch bản:

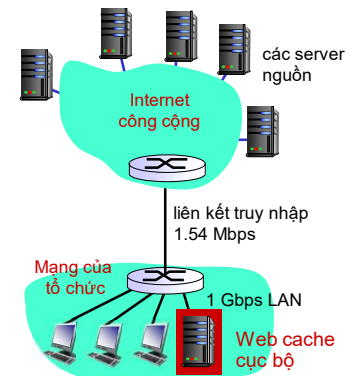
- ❖ Tốc độ liên kết truy nhập: 1.54 Mbps
- ❖ RTT từ bộ định tuyến của tổ chức đến server: 2 giây
- ❖ Kích thước đối tượng web: 100K bits
- ❖ Tốc độ yêu cầu trung bình từ trình duyệt đến server gốc: 15/giây
 - tốc độ dữ liệu trung bình tới trình duyệt: 1.50 Mbps

Chi phí: web cache (rẻ!)

Hiệu suất:

- ❖ Việc sử dụng LAN: ?
- ❖ Việc sử dụng liên kết truy nhập = ?
- ❖ trễ trung bình đầu cuối-đầu cuối = ?

Tính toán việc sử dụng liên kết, độ trễ như thế nào?

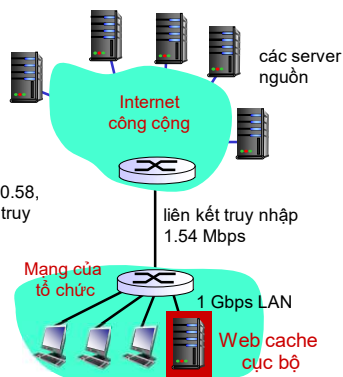


Tăng ứng dụng 2-41

Tính toán việc sử dụng liên kết truy nhập, trễ đầu cuối-đầu cuối với bộ đệm (cache)

Giả sử tỷ lệ hỗ trợ của cache là 0.4:

- ❖ 40% yêu cầu được cung cấp bởi cache, với độ trễ thấp (msec)
- ❖ 60% yêu cầu được cung cấp bởi server nguồn
 - tỷ lệ tới trình duyệt qua liên kết truy nhập = $0.6 * 1.50 \text{ Mbps} = 0.9 \text{ Mbps}$
 - Việc sử dụng liên kết truy nhập = $0.9 / 1.54 = 0.58$, nghĩa là trễ xếp hàng thấp (msec) tại liên kết truy nhập.
- ❖ Trễ trung bình đầu cuối-đầu cuối:
 - = $0.6 * (\text{trễ từ các server gốc})$
 - + $0.4 * (\text{trễ khi được cung cấp tại cache})$
 - = $0.6 * (2.01) + 0.4 * (\sim \text{msecs}) = \sim 1.2 \text{ secs}$

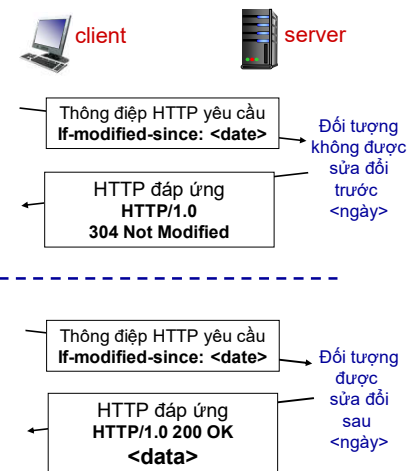


Trễ đầu cuối-đầu cuối trung bình thấp hơn so với liên kết 154 Mbps (và cũng rẻ hơn!)

Tăng ứng dụng 2-42

Trình duyệt đệm: GET có điều kiện

- ❖ **Mục tiêu:** không gửi đối tượng nếu trình duyệt đã có phiên bản được cập nhật trong bộ nhớ đệm
 - Không có trễ truyền đối tượng (hoặc sử dụng tài nguyên mạng).
- ❖ **Client:** xác định ngày của bản sao được lưu trong bộ nhớ đệm trong HTTP yêu cầu
 - If-modified-since: <date>**
- ❖ **server:** đáp ứng không chứa đối tượng nếu bản sao được lưu trong bộ nhớ đệm của trình duyệt đã được cập nhật:
 - HTTP/1.0 304 Not Modified**



Tăng ứng dụng 2-43

Chương 2: Nội dung

2.1. Nguyên lý của ứng dụng mạng

- 2.1.1. Các kiến trúc của ứng dụng mạng
- 2.1.2. Truyền thông giữa các tiến trình
- 2.1.3. Các dịch vụ giao vận

2.2. Web và HTTP

2.3. FTP

2.4. Thư điện tử

2.5. DNS (Domain Name Systems)

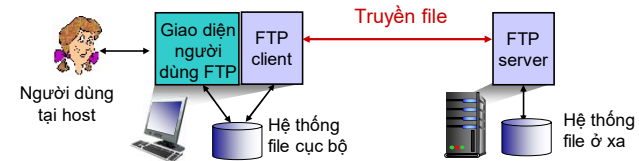
2.6. Ứng dụng Peer-to-peer

2.7. Video streaming và các mạng phân phối nội dung

2.8. Lập trình socket với UDP và TCP

Tăng ứng dụng 2-44

FTP: giao thức truyền file (file transfer protocol)

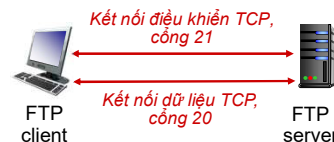


- ❖ Truyền file đến/từ host ở xa
- ❖ Mô hình client/server
 - **client**: phía khởi tạo việc truyền (đến/từ host ở xa)
 - **server**: host ở xa
- ❖ ftp: RFC 959
- ❖ ftp server: cổng 21

Tăng ứng dụng 2-45

FTP: kết nối dữ liệu và kết nối điều khiển riêng biệt nhau

- ❖ FTP client tiếp xúc với FTP server tại cổng số 21, sử dụng TCP
- ❖ client được cấp phép qua kết nối điều khiển
- ❖ client duyệt thư mục ở xa, gửi lệnh qua kết nối điều khiển
- ❖ Khi server nhận lệnh truyền file, **server** mở kết nối TCP thứ 2 (kết nối dữ liệu) (cho file) tới client
- ❖ Sau khi truyền một file, server đóng kết nối dữ liệu.



- ❖ server mở một kết nối dữ liệu khác để truyền một file khác.
- ❖ Kết nối điều khiển: **"out of band"**
- ❖ FTP server duy trì "trạng thái": thư mục hiện hành, sự cấp phép trước đó.

Tăng ứng dụng 2-46

Các lệnh và đáp ứng của FTP

Ví dụ các lệnh:

- ❖ Được gửi như các văn bản dạng mã ASCII qua kênh điều khiển
- ❖ **USER username**
- ❖ **PASS password**
- ❖ **LIST** trả về danh sách các file trong thư mục hiện hành
- ❖ **RETR filename** trích chọn (lấy) file
- ❖ **STOR filename** lưu (đặt) file vào host ở xa

Ví dụ các mã lệnh trả về

- ❖ Mã trạng thái và cụm từ trạng thái (như trong HTTP)
- ❖ **331 Username OK, password required**
- ❖ **125 data connection already open; transfer starting**
- ❖ **425 Can't open data connection**
- ❖ **452 Error writing file**

Tăng ứng dụng 2-47

Chương 2: Nội dung

2.1. Nguyên lý của ứng dụng mạng

- 2.1.1. Các kiến trúc của ứng dụng mạng
- 2.1.2. Truyền thông giữa các tiến trình
- 2.1.3. Các dịch vụ giao vận

2.2. Web và HTTP

2.3. FTP

2.4. Thư điện tử

2.5. DNS (Domain Name Systems)

2.6. Ứng dụng Peer-to-peer

2.7. Video streaming và các mạng phân phối nội dung

2.8. Lập trình socket với UDP và TCP

Tăng ứng dụng 2-48

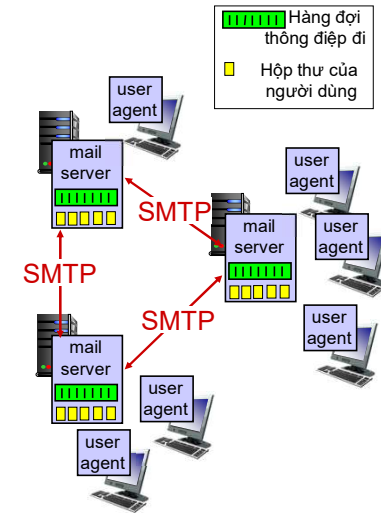
Thư điện tử

Ba thành phần chính:

- ❖ user agent
- ❖ mail server
- ❖ Giao thức truyền thư đơn giản (simple mail transfer protocol): SMTP

User Agent

- ❖ Còn được gọi là “mail reader”
- ❖ Soạn thảo, sửa, đọc các thông điệp thư
- ❖ Ví dụ: Outlook, iPhone mail client
- ❖ Các thông điệp đi/đến được lưu trên server



Tăng ứng dụng 2-49

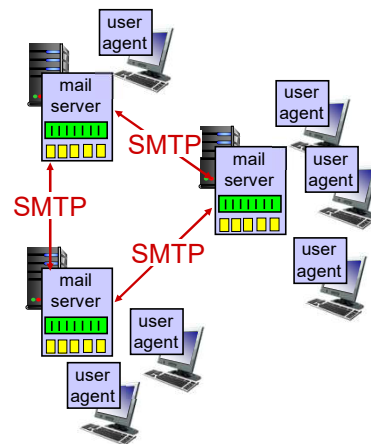
Thư điện tử: mail server

mail server:

- ❖ **Hộp thư (mailbox)** chứa các thông điệp thư đi đến người dùng
- ❖ **Hàng đợi thông điệp (message queue)** của các thông điệp thư đi ra ngoài (chuẩn bị được gửi đi)

Giao thức SMTP giữa các mail server thực hiện gửi các thông điệp thư

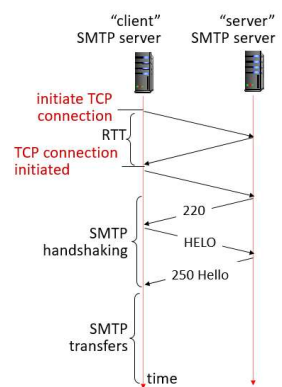
- ❖ client: gửi đến mail server
- ❖ server: mail server nhận



Tăng ứng dụng 2-50

Thư điện tử: SMTP [RFC 5321]

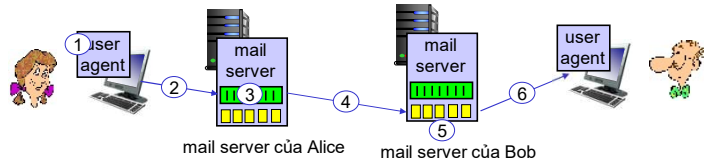
- ❖ Sử dụng TCP để truyền tin cậy thông điệp thư điện tử từ client đến server, cổng số 25
- ❖ Truyền trực tiếp: từ server gửi tới server nhận
- ❖ Truyền theo 3 bước
 - SMTP bắt tay (chào hỏi)
 - SMTP truyền thông điệp
 - SMTP đóng
- ❖ Tương tác lệnh/đáp ứng (như HTTP)
 - **Lệnh:** văn bản ASCII
 - **Đáp ứng:** mã trạng thái và các cụm từ trạng thái



Tăng ứng dụng 2-51

Kịch bản: Alice gửi thư tới Bob

- 1) Alice dùng UA soạn thảo thông điệp để "gửi tới" bob@someschool.edu
- 2) UA của Alice gửi thông điệp tới mail server của cô ấy; thông điệp được đặt trong hàng đợi thông điệp
- 3) Phía client của SMTP mở kết nối TCP tới mail server của Bob.
- 4) SMTP client gửi thông điệp của Alice qua kết nối TCP
- 5) mail server của Bob đặt thông điệp trong hộp thư của Bob
- 6) Bob kích hoạt user agent để đọc thông điệp thư



Tăng ứng dụng 2-52

Ví dụ tương tác SMTP

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

Tăng ứng dụng 2-53

Tự thử nghiệm tương tác SMTP:

- ❖ `telnet servername 25`
- ❖ Thấy 220 trả lời từ server
- ❖ Nhập các lệnh HELO, MAIL FROM, RCPT TO, DATA, QUIT

Các lệnh trên cho phép gửi email mà không cần dùng email client (reader).

Tăng ứng dụng 2-54

SMTP

- ❖ SMTP sử dụng kết nối bền vững
- ❖ SMTP yêu cầu thông điệp (phần tiêu đề & phần thân) ở dạng mã ASCII 7-bit
- ❖ SMTP server sử dụng CRLF.CRLF để xác định điểm kết thúc của thông điệp

So sánh với HTTP:

- ❖ HTTP: kéo
- ❖ SMTP: đẩy
- ❖ Cả hai đều sử dụng tương tác lệnh/đáp ứng, các mã trạng thái dạng bảng mã ASCII
 - HTTP: mỗi đối tượng được đóng gói trong thông điệp đáp ứng của nó.
 - SMTP: nhiều đối tượng được gửi trong thông điệp nhiều phần.

Tăng ứng dụng 2-55

Định dạng thông điệp thư

SMTP: giao thức trao đổi các thông điệp thư

RFC 2822: xác định *cú pháp* cho thông điệp email:

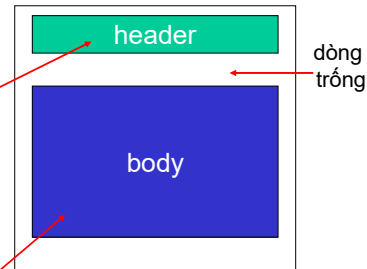
❖ Các dòng tiêu đề, ví dụ:

- To:
- From:
- Subject:

Những dòng này trong phần nội dung của thông điệp email, **khác** với các lệnh SMTP MAIL FROM, RCPT TO!

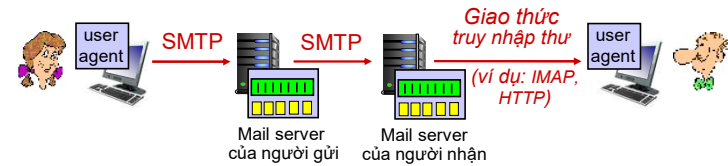
❖ Phần thân: “thông điệp”

- Chỉ dùng các ký tự mã ASCII



Tăng ứng dụng 2-56

Lấy thư: Giao thức truy nhập thư



❖ **SMTP**: phân phối/lưu trữ các thông điệp thư tới/tại server của người nhận

❖ Giao thức truy nhập thư: trích xuất thư từ server

- **POP**: Post Office Protocol [RFC 1939]: cấp phép, tải thư
- **IMAP**: Internet Mail Access Protocol [RFC 3501]: các thông điệp được lưu trữ trên server, IMAP cung cấp tính năng truy xuất, xóa, sắp xếp theo thư mục các thông điệp lưu trữ trên server.
- **HTTP**: gmail, Hotmail, Yahoo!Mail, v.v. cung cấp giao diện dựa trên web trên SMTP (để gửi), IMAP (hoặc POP) để truy xuất thư.

Tăng ứng dụng 2-57

Giao thức POP3

Giai đoạn cấp phép

- ❖ Các lệnh của client:
 - **user**: khai báo tên người dùng
 - **pass**: mật khẩu
- ❖ Các đáp ứng của server
 - **+OK**
 - **-ERR**

Giai đoạn giao dịch, client:

- ❖ **list**: liệt kê các số thông điệp
- ❖ **retr**: trích xuất thông điệp theo số
- ❖ **dele**: xóa
- ❖ **quit**

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on

C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

Tăng ứng dụng 2-58

POP3 và IMAP

POP3

- ❖ Ví dụ trước dùng POP3 với chế độ “tải và xóa”
 - Bob không thể đọc lại thư nếu thay đổi client
- ❖ POP3 với chế độ “tải và lưu giữ”: sao các thông điệp lên các client khác nhau
- ❖ POP3 không giữ trạng thái của các phiên làm việc

IMAP

- ❖ Lưu giữ tất cả các thông điệp tại một nơi là server
- ❖ Cho phép người dùng tổ chức các thông điệp theo dạng các thư mục
- ❖ Lưu giữ trạng thái của người dùng qua các phiên làm việc:
 - Đặt tên thư mục và ánh xạ giữa các ID của thông điệp với tên thư mục

Tăng ứng dụng 2-59

Chương 2: Nội dung

2.1. Nguyên lý của ứng dụng mạng

- 2.1.1. Các kiến trúc của ứng dụng mạng
- 2.1.2. Truyền thông giữa các tiến trình
- 2.1.3. Các dịch vụ giao vận

2.2. Web và HTTP

2.3. FTP

2.4. Thư điện tử

2.5. DNS (Domain Name Systems)

2.6. Ứng dụng Peer-to-peer

2.7. Video streaming và các mạng phân phối nội dung

2.8. Lập trình socket với UDP và TCP

Tăng ứng dụng 2-60

DNS: Hệ thống tên miền (domain name system)

Con người: có nhiều định danh:

- CMT, tên, số hộ chiếu

Các host và router trên Internet:

- Địa chỉ IP (32 bit) – được dùng để gán địa chỉ cho các gói tin
- “tên miền”, ví dụ: www.yahoo.com – được con người sử dụng

Hỏi: Làm cách nào để ánh xạ giữa địa chỉ IP và tên miền, và ngược lại?

Hệ thống tên miền:

- ❖ *Cơ sở dữ liệu phân tán* được cài đặt phân cấp với nhiều **server tên miền**
- ❖ *Giao thức tầng ứng dụng*: để các host, các server tên miền truyền thông được thì cần phải **phân giải** các tên miền (diễn dịch địa chỉ/tên miền)
 - Chú ý: chức năng lõi của Internet, được cài đặt như giao thức tầng ứng dụng
 - Phức tạp tại “phần cạnh” của mạng

Tăng ứng dụng 2-61

DNS: các dịch vụ và cấu trúc

Các dịch vụ của DNS

- ❖ Dịch tên host thành địa chỉ IP
- ❖ Bí danh của host
 - Các tên đúng chuẩn, các tên là bí danh
- ❖ Bí danh mail server
- ❖ Phân tán tải
 - Nhân rộng các máy chủ Web: nhiều địa chỉ IP tương ứng với một tên

Tại sao không tập trung hóa trong DNS?

- ❖ Một điểm chịu lỗi
- ❖ Lưu lượng
- ❖ Cơ sở dữ liệu tập trung ở xa
- ❖ Bảo trì

Trả lời: Không thể thực hiện với quy mô lớn!

- Chỉ riêng máy chủ DNS Comcast: 600 tỷ truy vấn DNS/ngày
- Riêng máy chủ DNS Akamai: 2,2T truy vấn DNS/ngày

Tăng ứng dụng 2-62

Một số vấn đề của DNS

Cơ sở dữ liệu phân tán khổng lồ:

- ~ tỷ bản ghi, mỗi bản ghi đơn giản.

Xử lý hàng nghìn tỷ truy vấn mỗi ngày:

- Số lượng truy vấn đọc *nhiều hơn* truy vấn ghi
- *Các vấn đề về hiệu năng*: hầu hết mọi giao dịch Internet đều tương tác với DNS – tính theo mili giây!

Không tập trung về mặt tổ chức và vật lý:

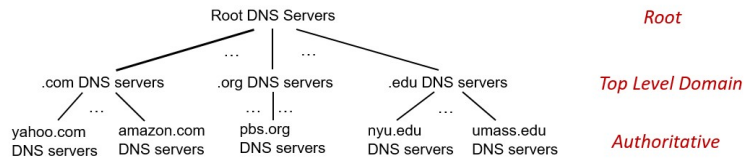
- hàng triệu tổ chức khác nhau chịu trách nhiệm về bản ghi của họ

Vấn đề: độ tin cậy, bảo mật



Tăng ứng dụng 2-63

DNS: cơ sở dữ liệu phân tán và phân cấp



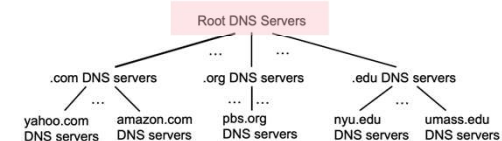
client muốn có địa chỉ IP cho www.amazon.com:

- ❖ client truy vấn server gốc để tìm com DNS server
- ❖ client truy vấn .com DNS server để lấy amazon.com DNS server
- ❖ client truy vấn amazon.com DNS server để lấy địa chỉ IP của www.amazon.com

Tăng ứng dụng 2-64

DNS: Các server tên gốc

- ❖ Là nơi liên hệ cuối cùng, chính thức, khi mà các server tên miền không thể phân giải tên.



Tăng ứng dụng 2-65

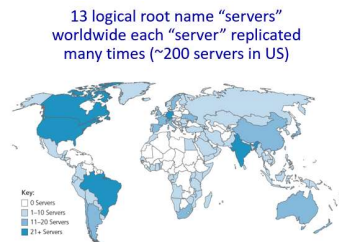
DNS: Các server tên gốc

- ❖ Là nơi liên hệ cuối cùng, chính thức, khi mà các server tên miền không thể phân giải tên.

- ❖ Chức năng Internet **vô cùng quan trọng**

- Internet không thể hoạt động nếu thiếu nó
- DNSSEC – cung cấp bảo mật (xác thực, tính toàn vẹn của thông điệp)

- ❖ ICANN (Internet Corporation for Assigned Names and Numbers) quản lý tên miền DNS gốc

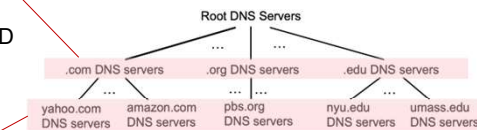


Tăng ứng dụng 2-66

Các server TLD và server có thẩm quyền

Các server top-level domain (TLD) :

- Chịu trách nhiệm cho tên miền .com, .org, .net, .edu, .aero, .jobs, .museums, và tất cả các tên miền cấp cao nhất thuộc quốc gia như: .cn, .uk, .fr, .ca, .jp
- Network Solutions: cơ quan đăng ký có thẩm quyền cho .com, .net TLD
- Educause: .edu TLD



Các server DNS có thẩm quyền:

- (Các) máy chủ DNS của chính tổ chức, cung cấp tên máy chủ có thẩm quyền để ánh xạ IP cho các máy chủ được đặt tên của tổ chức
- có thể được duy trì bởi tổ chức hoặc nhà cung cấp dịch vụ

Tăng ứng dụng 2-67

Server tên DNS cục bộ

- Khi máy chủ thực hiện truy vấn DNS, nó sẽ được gửi đến máy chủ DNS *cục bộ* của nó
 - Máy chủ DNS cục bộ trả lời:
 - từ thông tin có trong bộ đệm cục bộ của các cặp tên-địa chỉ (có thể đã bị cũ!)
 - chuyển tiếp yêu cầu vào hệ thống phân cấp DNS để giải quyết
 - Mỗi ISP có máy chủ tên DNS cục bộ:
 - MacOS: `% scutil --dns`
 - Windows: `>ipconfig /all`
- Máy chủ DNS cục bộ không hoàn toàn thuộc về hệ thống phân cấp.

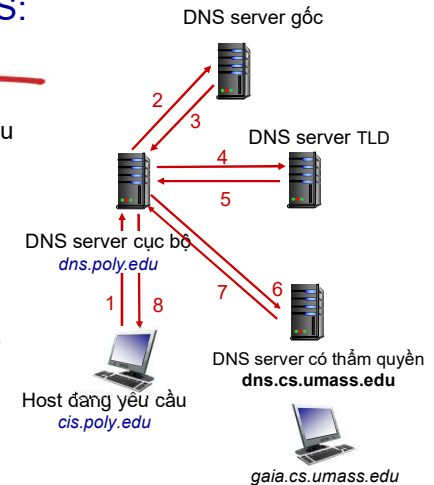
Tăng ứng dụng 2-68

Phân giải tên DNS: Truy vấn lặp

- ❖ Ví dụ: Host tại cis.poly.edu muốn lấy địa chỉ IP của gaia.cs.umass.edu

Truy vấn lặp:

- ❖ Server được hỏi sẽ trả lời với tên của server sẽ có thể hỏi được tiếp
- ❖ “Tôi không biết tên đó, nhưng có thể hỏi server này”



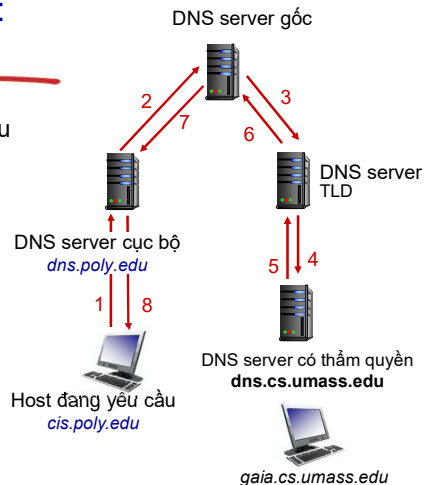
Tăng ứng dụng 2-69

Phân giải tên DNS: Truy vấn đệ quy

- ❖ Ví dụ: Host tại cis.poly.edu muốn lấy địa chỉ IP của gaia.cs.umass.edu

Truy vấn đệ quy:

- Đặt trách nhiệm phân giải tên lên server tên được hỏi
- Tải nặng tại các cấp cao hơn trong hệ thống phân cấp?



Tăng ứng dụng 2-70

DNS: lưu và cập nhật các bản ghi

- ❖ Khi server tên học cách ánh xạ, nó sẽ *lưu* ánh xạ vào bộ nhớ đệm, và ngay lập tức sẽ phản hồi truy vấn với ánh xạ này.
 - Việc lưu trong bộ đệm sẽ giúp cải thiện thời gian phản hồi
 - Mục lưu sẽ bị quá hạn (bị xóa) sau một thời gian (TTL)
 - Các server TLD thường được lưu trong các server tên cục bộ
 - Do đó, các server tên gốc sẽ không thường xuyên được truy cập
- ❖ Các mục được lưu trong bộ đệm có thể bị *quá hạn*
 - Nếu host thay đổi địa chỉ IP, thì Internet sẽ có thể không biết được cho đến khi TTL hết hạn
 - Sử dụng cơ chế best effort để dịch tên-địa chỉ!

Tăng ứng dụng 2-71

Bản ghi DNS (DNS records)

DNS: cơ sở dữ liệu phân tán lưu trữ các bản ghi nguồn (resource records - RR)

Định dạng RR: (name, value, type, ttl)

type=A

- **name** là tên máy (Tên miền)
- **Value** là địa chỉ IP

type=NS

- **name** là tên miền (ví dụ: foo.com)
- **value** là tên host của server có thẩm quyền cho tên miền này

type=CNAME

- **name** là tên bí danh của tên "chuẩn" (tên thật)
- **www.ibm.com** tên thật là **servereast.backup2.ibm.com**
- **value** là tên chuẩn

type=MX

- **value** là tên của mail server được kết hợp với **name**

Tăng ứng dụng 2-72

Giao thức, thông điệp DNS

- ❖ Các thông điệp **truy vấn** và **trả lời** đều có cùng **định dạng thông điệp**

Tiêu đề thông điệp

- ❖ **identification:** 16 bit cho truy vấn/trả lời
- ❖ **flags:**
 - Truy vấn hoặc trả lời
 - Độ quy chờ
 - Độ quy sẵn sàng
 - Trả lời có thẩm quyền

← 2 bytes →		← 2 bytes →	
identification	flags		
# questions	# answer RRs		
# authority RRs	# additional RRs		
questions (variable # of questions)			
answers (variable # of RRs)			
authority (variable # of RRs)			
additional info (variable # of RRs)			

Tăng ứng dụng 2-73

Giao thức, thông điệp DNS

		← 2 bytes	→ ← 2 bytes	→
		identification	flags	
		# questions	# answer RRs	
		# authority RRs	# additional RRs	
Trường tên, loại truy vấn	→	questions (variable # of questions)		
RR trong trả lời cho truy vấn	→	answers (variable # of RRs)		
Bản ghi của server có thẩm quyền	→	authority (variable # of RRs)		
Thông tin "hữu ích" có thể được sử dụng	→	additional info (variable # of RRs)		

Trường tên, loại truy vấn

RR trong trả lời cho truy vấn

Bản ghi của server có thẩm quyền

Thông tin "hữu ích" có thể được sử dụng

Tăng ứng dụng 2-74

Đưa thông tin vào trong DNS

- ❖ Ví dụ: Tạo mới "Network Utopia"
- ❖ Đăng ký tên miền networkutopia.com tại **DNS registrar** (Ví dụ: Network Solutions)
 - Cung cấp tên, địa chỉ IP của server tên có thẩm quyền (sơ cấp và thứ cấp)
 - registrar chèn 2 bản ghi NS, A vào server .com TLD:
 - (networkutopia.com, dns1.networkutopia.com, NS)
 - (dns1.networkutopia.com, 212.212.212.1, A)
- ❖ Tạo server có thẩm quyền cục bộ với địa chỉ IP 212.212.212.1
 - Bản ghi loại A cho www.networkutopia.com
 - Bản ghi loại MX cho networkutopia.com

Tăng ứng dụng 2-75

Bảo mật DNS

Tấn công DDoS

- ❖ Tấn công server gốc với lưu lượng lớn
 - Không thành công cho đến nay
 - Lọc lưu lượng
 - DNS server cục bộ cache IP của TLD server, cho phép bỏ qua server gốc
- ❖ Tấn công TLD servers với lưu lượng lớn
 - Tiềm tàng nhiều nguy cơ hơn

Tấn công giả mạo

- ❖ Chặn các truy vấn DNS, trả về các câu trả lời không có thật
- ❖ Đầu độc bộ đệm DNS
 - Gửi trả lời giả mạo đến DNS server, mà các server này có thể cache lại

RFC 4033: Dịch vụ xác thực DNSSEC

Chương 2: Nội dung

2.1. Nguyên lý của ứng dụng mạng

- 2.1.1. Các kiến trúc của ứng dụng mạng
- 2.1.2. Truyền thông giữa các tiến trình
- 2.1.3. Các dịch vụ giao vận

2.2. Web và HTTP

2.3. FTP

2.4. Thư điện tử

2.5. DNS (Domain Name Systems)

2.6. Ứng dụng Peer-to-peer

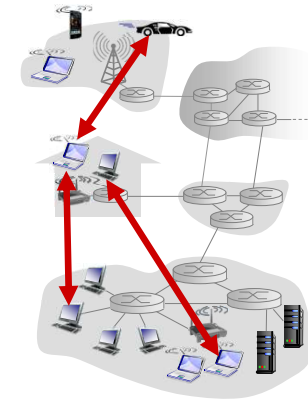
2.7. Video streaming và các mạng phân phối nội dung

2.8. Lập trình socket với UDP và TCP

Tăng ứng dụng 2-77

Kiến trúc mạng P2P thuần túy

- ❖ Không cần phải có server luôn hoạt động
- ❖ Các hệ thống cuối tùy ý kết nối trực tiếp
- ❖ Các peer (các nút mạng) yêu cầu dịch vụ từ các peer khác, cung cấp dịch vụ để đáp ứng lại các peer khác.
 - Khả năng tự mở rộng - các nút cung cấp khả năng dịch vụ mới và nhu cầu dịch vụ mới.
- ❖ Các peer không cần kết nối liên tục vào hệ thống mạng và có thể thay đổi địa chỉ IP.
 - Quản lý phức tạp



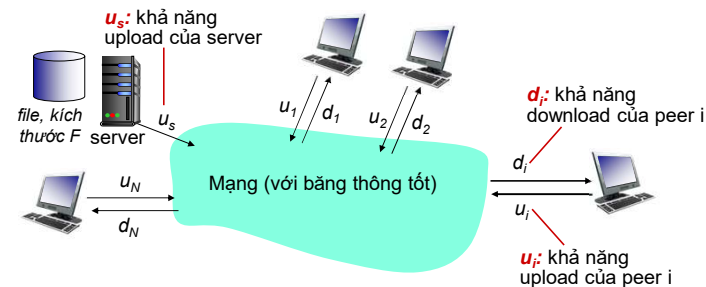
Ví dụ: Chia sẻ file (BitTorrent); Streaming (KanKan); VoIP (Skype)

Tăng ứng dụng 2-78

Chia sẻ file: client-server và P2P

Hỏi: Cần mất bao lâu để truyền (phân phối) file (có kích thước F) từ một server đến N peer?

- Khả năng upload/download (tải lên/tải về) của peer bị giới hạn bởi tài nguyên.



Tăng ứng dụng 2-79

Thời gian truyền file: client-server

- ❖ **Việc truyền của server:** phải gửi tuần tự (upload) N bản sao của file:

- Thời gian gửi một bản sao: F/u_s
- Thời gian gửi N bản sao: NF/u_s

- ❖ **Client:** Mỗi client phải download bản sao của file

- $d_{\min}$ = tốc độ download nhỏ nhất của client
- Thời gian download (ứng với $d_{\min}$) của client: $F/d_{\min}$

Thời gian để phân phối F tới N client sử dụng cách tiếp cận client-server

$$D_{c-s} \geq \max\{NF/u_s, F/d_{\min}\}$$

Tăng tuyến tính theo N

Tăng ứng dụng 2-80

Thời gian truyền file: P2P

- ❖ **Việc truyền của server:** phải upload ít nhất một bản sao của file:

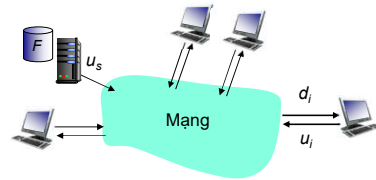
- Thời gian để gửi một bản: F/u_s

- ❖ **Client:** mỗi client phải download bản sao của file

- Thời gian download (ứng với d_{min}) của client: F/d_{min}

- ❖ **Các client:** phải download NF bits

- Tốc độ upload cao nhất (giới hạn) là $u_s + \sum u_i$



Thời gian để phân phối F

tới N client sử dụng
cách tiếp cận P2P

$$D_{P2P} \geq \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$$

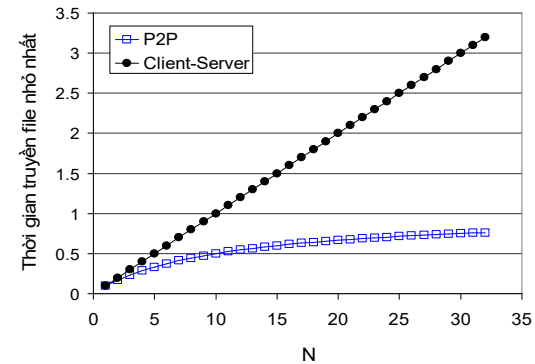
Tăng tuyến tính theo N ...

... nhưng để thực hiện việc này, mỗi peer phải có khả năng phục vụ

Tăng ứng dụng 2-81

Ví dụ so sánh client-server với P2P

Tốc độ upload của client = u , $F/u = 1$ giờ, $u_s = 10u$, $d_{min} \geq u_s$

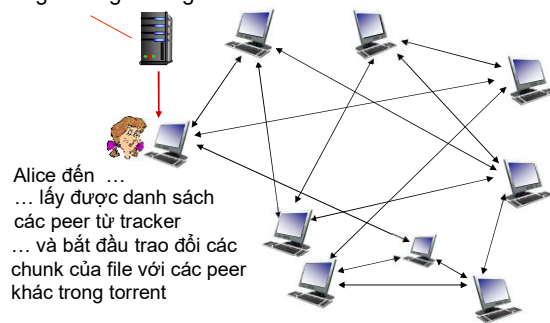


Truyền file P2P: BitTorrent

- ❖ File được chia thành các chunk (phần) có kích thước là 256Kb
- ❖ Các peer trong torrent gửi/nhận các chunk của file

tracker: kiểm tra các peer đang tham gia trong torrent

torrent: nhóm các peer trao đổi các chunk của một file



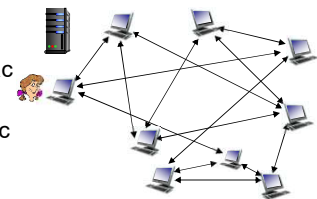
Alice đến ...
... lấy được danh sách các peer từ tracker
... và bắt đầu trao đổi các chunk của file với các peer khác trong torrent

Tăng ứng dụng 2-83

Truyền file P2P: BitTorrent

- ❖ peer tham gia vào torrent:

- Không có các chunk, nhưng sẽ tích lũy chúng qua thời gian từ các peer khác
- Đăng ký với tracker để nhận được danh sách các peer, kết nối với tập con của các peer ("các hàng xóm")



- ❖ Trong khi download, peer sẽ upload các chunk tới các peer khác
- ❖ peer có thể thay đổi các peer mà nó sẽ trao đổi các chunk
- ❖ **churn:** các peer có thể đến hoặc đi
- ❖ Khi peer có được toàn bộ file, nó có thể (ích kỷ) rời đi hoặc (vị tha) ở lại trong torrent

Tăng ứng dụng 2-84

BitTorrent: yêu cầu lấy, gửi các chunk của file

Yêu cầu lấy chunk:

- ❖ Tại bất kỳ thời điểm nào, các peer khác nhau đều có các tập con các chunk khác nhau của file.
- ❖ Theo định kỳ, Alice sẽ hỏi mỗi peer về danh sách các chunk mà họ có
- ❖ Alice sẽ yêu cầu các chunk còn thiếu từ các peer, phần hiếm nhất trước

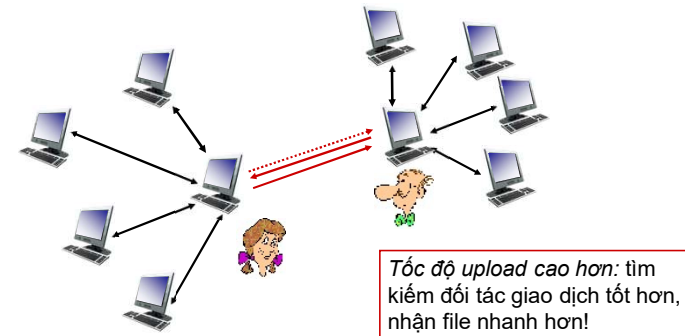
Gửi chunk: tit-for-tat

- ❖ Alice gửi các chunk tới 4 peer hiện đang gửi chunk của cô ấy với **tốc độ cao nhất**
 - Các peer khác đang bị chặn bởi Alice (không nhận được chunk từ cô ấy)
 - Đánh giá lại top 4 sau mỗi 10 giây
- ❖ Cứ mỗi 30 giây: chọn các peer khác một cách ngẫu nhiên, và bắt đầu gửi chunk
 - “Tối ưu hóa không chặn” peer này
 - Peer được chọn mới sẽ được gia nhập vào top 4

Tăng ứng dụng 2-85

BitTorrent: tit-for-tat

- (1) Alice “tối ưu hóa không chặn” Bob
- (2) Alice trở thành một nhà cung cấp trong top 4 của Bob;
- (3) Ngược lại, Bob trở thành một nhà cung cấp trong top 4 của Alice



Tăng ứng dụng 2-86

Chương 2: Nội dung

2.1. Nguyên lý của ứng dụng mạng

- 2.1.1. Các kiến trúc của ứng dụng mạng
- 2.1.2. Truyền thông giữa các tiến trình
- 2.1.3. Các dịch vụ giao vận

2.2. Web và HTTP

2.3. FTP

2.4. Thư điện tử

2.5. DNS (Domain Name Systems)

2.6. Ứng dụng Peer-to-peer

2.7. Video streaming và các mạng phân phối nội dung

2.8. Lập trình socket với UDP và TCP

Tăng ứng dụng 2-87

Video Streaming và CDNs: ngữ cảnh

- Lưu lượng streaming video: tiêu tốn lượng lớn băng thông Internet
 - Netflix, YouTube, Amazon Prime: 80% lưu lượng truy nhập ISP khu vực dân cư (2020)
- **Khó khăn:** quy mô – làm thế nào để tiếp cận ~1B người dùng?
- **Khó khăn:** tính không đồng nhất
 - Người dùng khác nhau có những khả năng khác nhau (ví dụ, có dây hoặc di động; giàu băng thông hay nghèo băng thông)
- **Giải pháp:** cơ sở hạ tầng mức ứng dụng, được phân tán



Tăng ứng dụng 2-88

Đa phương tiện: video

- Video: chuỗi hình ảnh được hiển thị ở tốc độ không đổi
 - Ví dụ: 24 hình ảnh/giây
- Hình ảnh kỹ thuật số: mảng pixel
 - mỗi pixel được biểu thị bằng bit
- Mã hóa: sử dụng sự dư thừa **bên trong** và **giữa** các hình ảnh để giảm số bit được sử dụng để mã hóa hình ảnh
 - Không gian (bên trong ảnh)
 - Thời gian (từ một hình ảnh tới hình ảnh tiếp theo)



Tăng ứng dụng 2-89

Đa phương tiện: video

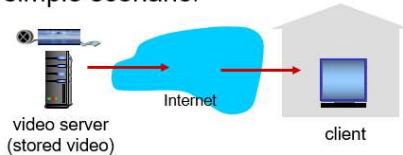
- CBR: (tốc độ bit không đổi):** tốc độ mã hóa video cố định
- VBR: (tốc độ bit thay đổi):** tốc độ mã hóa video thay đổi khi lượng mã hóa theo không gian, thời gian thay đổi
- Ví dụ:**
 - MPEG 1 (CD-ROM) 1.5 Mbps
 - MPEG2 (DVD) 3-6 Mbps
 - MPEG4 (thường được sử dụng trong Internet, 64Kbps – 12 Mbps)



Tăng ứng dụng 2-90

Streaming video được lưu trữ

simple scenario:

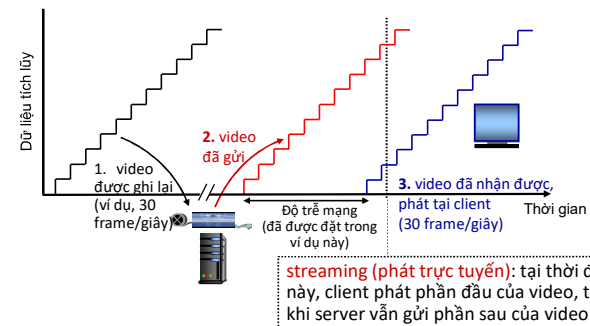


Những khó khăn chính:

- Băng thông từ server-đến-client sẽ **thay đổi** theo thời gian, với mức độ tắc nghẽn mạng thay đổi (trong nhà, mạng truy nhập, lỗi mạng, máy chủ video)
- Mất gói, trễ do tắc nghẽn sẽ làm chậm quá trình phát hoặc dẫn đến chất lượng video kém

Tăng ứng dụng 2-91

Streaming video được lưu trữ



Tăng ứng dụng 2-92

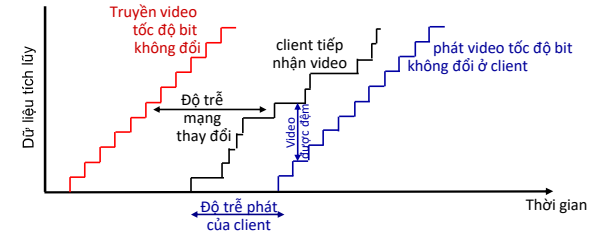
Streaming video được lưu trữ: khó khăn

- **Ràng buộc phát liên tục:** trong khi client phát video, thời gian phát phải khớp với thời gian ban đầu
 - ... nhưng **độ trễ của mạng có thể thay đổi** (jitter), do đó sẽ cần **bộ đệm phía client** để phù hợp với ràng buộc phát liên tục
- Những khó khăn khác:
 - Tương tác client: tạm dừng, tua nhanh, tua lại, chuyển qua video
 - Các gói tin video có thể bị mất, được truyền lại



Tăng ứng dụng 2-93

Streaming video được lưu trữ: đệm phát



- **Bộ đệm phía client và độ trễ phát:** bù cho độ trễ do mạng thêm vào, độ trễ jitter

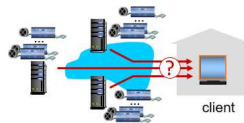
Tăng ứng dụng 2-94

Streaming đa phương tiện: DASH

Dynamic, Adaptive
Streaming over HTTP

server:

- Chia tệp video thành nhiều đoạn
- Mỗi đoạn được mã hóa ở nhiều tốc độ khác nhau
- Mã hóa tốc độ khác nhau được lưu trữ trong các tệp khác nhau
- Các tệp được sao chép trong các nút CDN khác nhau
- **Tệp kê khai:** cung cấp URL cho các đoạn khác nhau.



client:

- Ước tính định kỳ bằng thông từ server-đến-client
- Tư vấn kê khai, yêu cầu từng đoạn tại từng thời điểm
 - Chọn tốc độ mã hóa tối đa bền vững với băng thông hiện tại
 - Có thể chọn tốc độ mã hóa khác nhau tại các thời điểm khác nhau (tùy thuộc vào băng thông có sẵn tại thời điểm đó) và từ các server khác nhau

Tăng ứng dụng 2-95

Streaming đa phương tiện: DASH

- **“thông minh” tại client:** client xác định

- **Khi nào** cần yêu cầu đoạn (để không xảy ra tình trạng thiếu bộ đệm hoặc tràn bộ đệm)
- **Yêu cầu tốc độ mã hóa nào** (chất lượng cao hơn khi có nhiều băng thông hơn)
- **Nơi** yêu cầu đoạn (có thể yêu cầu từ URL server “gần” với client hoặc có băng thông khả dụng cao)



Streaming video = mã hóa + DASH + đệm phát

Tăng ứng dụng 2-96

Các mạng phân phối nội dung (CDNs)

Khó khăn: làm thế nào để truyền tải nội dung (được chọn lọc từ hàng triệu video) tới hàng trăm nghìn người dùng cùng lúc?

- **Lựa chọn 1:** “mega-server” đơn, lớn
 - Một điểm chịu lỗi
 - Điểm tắc nghẽn mạng
 - Đường đi dài (và có thể bị tắc nghẽn) tới các client ở xa

....quá đơn giản: giải pháp này **không mở rộng được quy mô**.

Tăng ứng dụng 2-97

Các mạng phân phối nội dung (CDNs)

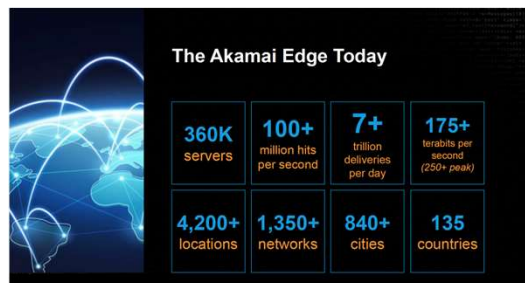
Khó khăn: làm thế nào để truyền tải nội dung (được chọn lọc từ hàng triệu video) tới hàng trăm nghìn người dùng cùng lúc?

- **Lựa chọn 2:** lưu trữ/phục vụ nhiều bản sao video tại nhiều trang web được phân phối theo địa lý
 - **Đi vào sâu (enter deep):** đẩy các server CDN vào sâu bên trong nhiều mạng truy nhập
 - Gần với người dùng
 - Akamai: 240,000 server được triển khai tại > 120 quốc gia (2015)
 - **Mang về nhà (bring home):** số lượng nhỏ hơn (khoảng 10) cụm lớn hơn trong các POP gần các mạng truy nhập
 - Được sử dụng bởi Limelight



Tăng ứng dụng 2-98

Akamai ngày nay:

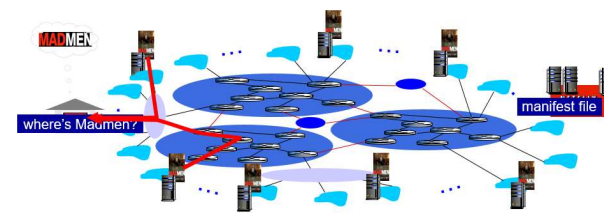


Nguồn: <https://networkingchannel.eu/living-on-the-edge-for-a-quarter-century-an-akamai-retrospective-downloads/>

Tăng ứng dụng 2-99

Netflix hoạt động như thế nào?

- Netflix: lưu trữ các bản sao nội dung (ví dụ: MADMEN) tại các nút OpenConnect CDN (trên toàn thế giới) của nó
- Thuê bao yêu cầu nội dung, nhà cung cấp dịch vụ trả về bảng kê khai
 - bằng cách sử dụng bảng kê khai, khách hàng sẽ truy xuất nội dung ở tốc độ có thể hỗ trợ cao nhất
 - có thể chọn tốc độ khác hoặc sao chép nếu đường dẫn mạng bị tắc nghẽn



Tăng ứng dụng 2-100

Các mạng phân phối nội dung (CDNs)



Khó khăn OTT: ứng phó với tình trạng tắc nghẽn Internet từ "biên"

- Nội dung nào cần đặt trong nút CDN nào?
- Từ nút CDN nào để truy xuất nội dung? Ở tốc độ nào?

Chương 2: Nội dung

2.1. Nguyên lý của ứng dụng mạng

- 2.1.1. Các kiến trúc của ứng dụng mạng
- 2.1.2. Truyền thông giữa các tiến trình
- 2.1.3. Các dịch vụ giao vận

2.2. Web và HTTP

2.3. FTP

2.4. Thư điện tử

2.5. DNS (Domain Name Systems)

2.6. Ứng dụng Peer-to-peer

2.7. Video streaming và các mạng phân phối nội dung

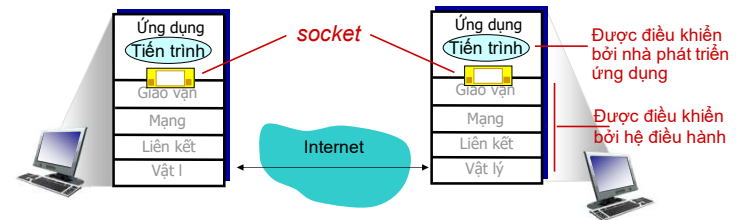
2.8. Lập trình socket với UDP và TCP

Tầng ứng dụng 2-102

Lập trình Socket

Mục đích: hiểu được cách xây dựng ứng dụng truyền thông client/server dùng socket

socket: là cánh cửa giữa các tiến trình ứng dụng và giao thức giao vận end-to-end



Tầng ứng dụng 2-103

Lập trình Socket

Hai loại socket cho hai dịch vụ tầng giao vận:

- **UDP:** truyền các gói tin không tin cậy
- **TCP:** truyền tin cậy, truyền dòng byte có hướng

Ví dụ ứng dụng:

1. Client đọc vào một dòng ký tự (dữ liệu) từ bàn phím và gửi dữ liệu đến server.
2. Server nhận dữ liệu và chuyển các ký tự thành dạng ký tự viết hoa.
3. Server gửi dữ liệu đã được chuyển thành dạng viết hoa về cho client.
4. Client nhận dữ liệu và hiển thị dòng ký tự lên màn hình.

Tầng ứng dụng 2-104

Lập trình socket với UDP

UDP: không có “kết nối” giữa client & server

- ❖ Không bắt tay trước khi gửi dữ liệu
- ❖ Bên gửi gán địa chỉ IP và số hiệu cổng đích vào trong mỗi gói tin
- ❖ Bên nhận sẽ trích địa chỉ IP và số hiệu cổng của bên gửi từ gói tin nhận được

UDP: dữ liệu được truyền có thể bị mất hoặc không đúng trình tự khi nhận

Quan điểm ứng dụng:

- ❖ UDP cung cấp truyền không tin cậy theo các nhóm byte (“các gói tin”) giữa client và server

Tầng ứng dụng 2-105

Tương tác client/server socket: UDP

server (chạy trên serverIP)

Tạo socket, port= x:
`serverSocket =
socket(AF_INET, SOCK_DGRAM)`

Đọc datagram từ
`serverSocket`

Ghi trả lời vào
`serverSocket`
địa chỉ client,
số hiệu cổng
cụ thể

client

Tạo socket:
`clientSocket =
socket(AF_INET, SOCK_DGRAM)`

Tạo datagram với IP của server
và port=x; gửi datagram qua
`clientSocket`

Đọc datagram từ
`clientSocket`

Đóng
`clientSocket`

Tăng ứng dụng 2-106

Ví dụ: UDP client

Python UDPClient

```
include thư viện socket của Python → from socket import *
serverName = 'hostname'
serverPort = 12000

Tạo UDP socket tới server → clientSocket = socket(socket.AF_INET,
                                                    socket.SOCK_DGRAM)

Nhận đầu vào từ bàn phím của người dùng → message = raw_input('Input lowercase sentence:')
Gắn tên, số cổng server vào message; gửi vào socket → clientSocket.sendto(message, (serverName, serverPort))

Đọc các ký tự trả lời từ socket vào chuỗi → modifiedMessage, serverAddress =
                                                    clientSocket.recvfrom(2048)

In ra chuỗi nhận được và đóng socket → print modifiedMessage
                                                    clientSocket.close()
```

Tăng ứng dụng 2-107

Ví dụ: UDP server

Python UDPServer

```
from socket import *
serverPort = 12000
```

Tạo UDP socket → `serverSocket = socket(AF_INET, SOCK_DGRAM)`

Liên kết socket với số cổng cục bộ 12000 → `serverSocket.bind(('', serverPort))`
`print "The server is ready to receive"`

Vòng lặp → `while 1:`

Đọc từ UDP socket vào message, lấy địa chỉ của client (IP và cổng của client) → `message, clientAddress = serverSocket.recvfrom(2048)`
`modifiedMessage = message.upper()`

Gửi chuỗi ký tự viết hoa cho client này → `serverSocket.sendto(modifiedMessage, clientAddress)`

Tăng ứng dụng 2-108

Lập trình socket với TCP

client phải tiếp xúc với server

- ❖ Tiến trình server phải chạy trước
- ❖ server phải tạo socket (cửa) để đón client tiếp xúc

client tiếp xúc với server bằng cách:

- ❖ Tạo TCP socket, xác định địa chỉ IP, số hiệu cổng của tiến trình server
- ❖ **Khi client tạo socket:** TCP client sẽ thiết lập kết nối tới TCP server

- ❖ Khi được tiếp xúc bởi client, **TCP server sẽ tạo socket mới** cho tiến trình server để truyền thông với client
 - Cho phép server "nói chuyện" với nhiều client
 - Các số hiệu cổng nguồn được dùng để phân biệt các client.

Quan điểm ứng dụng:

TCP cung cấp truyền tin cậy, truyền dòng byte theo đúng trình tự giữa client và server.

Tăng ứng dụng 2-109

Tương tác client/server socket: TCP

server (chạy trên `hostid`)

Tạo socket,
port=`x`, cho yêu cầu
đến:
`serverSocket = socket()`

Đợi yêu cầu kết
nối đến
`connectionSocket =
serverSocket.accept()`

Đọc yêu cầu từ
`connectionSocket`

Ghi trả lời vào
`connectionSocket`

Đóng
`connectionSocket`

client

Tạo socket,
kết nối tới `hostid`, port=`x`
`clientSocket = socket()`

Gửi yêu cầu dùng
`clientSocket`

Đọc trả lời từ
`clientSocket`

Đóng
`clientSocket`

TCP
Thiết lập kết nối

Tăng ứng dụng 2-110

Ví dụ: TCP client

Python TCPClient

```
from socket import *  
serverName = 'servername'  
serverPort = 12000  
clientSocket = socket(AF_INET, SOCK_STREAM)  
clientSocket.connect((serverName, serverPort))  
sentence = raw_input('Input lowercase sentence:')  
clientSocket.send(sentence)  
modifiedSentence = clientSocket.recv(1024)  
print 'From Server:', modifiedSentence  
clientSocket.close()
```

Tạo TCP socket tới
server, remote port 12000

Không cần gán tên, số
cổng server

Tăng ứng dụng 2-111

Ví dụ: TCP server

Python TCPServer

```
from socket import *  
serverPort = 12000  
serverSocket = socket(AF_INET, SOCK_STREAM)  
serverSocket.bind(('', serverPort))  
serverSocket.listen(1)  
print 'The server is ready to receive'  
while 1:  
    connectionSocket, addr = serverSocket.accept()  
    sentence = connectionSocket.recv(1024)  
    capitalizedSentence = sentence.upper()  
    connectionSocket.send(capitalizedSentence)  
    connectionSocket.close()
```

Tạo socket TCP
chờ

server bắt đầu lắng nghe
các yêu cầu TCP đi đến

Vòng lặp

server chờ `accept()` cho các
yêu cầu đến, socket mới
được tạo khi trả về

Đọc các byte từ socket (nhưng
không có địa chỉ như trong UDP)

Đóng kết nối tới client
này

Tăng ứng dụng 2-112

Xây dựng một Web server đơn giản

- ❖ Quản lý một yêu cầu HTTP
- ❖ Chấp nhận yêu cầu
- ❖ Phân tích cú pháp phần tiêu đề (header)
- ❖ Lấy file được yêu cầu từ hệ thống file của server
- ❖ Tạo thông điệp đáp ứng HTTP:
 - các dòng header + file
- ❖ Gửi đáp ứng đến client
- ❖ Sau khi tạo server, có thể yêu cầu file dùng trình duyệt (ví dụ: IE)
- ❖ Xem giáo trình để biết thêm chi tiết

Tăng ứng dụng 2-113

Chương 2: Tổng kết

Trình bày các vấn đề liên quan đến ứng dụng mạng!

- ❖ Kiến trúc của ứng dụng
 - client-server
 - P2P
- ❖ Các yêu cầu dịch vụ của ứng dụng:
 - Truyền tin cậy, băng thông, trễ
- ❖ Mô hình dịch vụ giao vận của Internet
 - Hướng kết nối, truyền tin cậy: TCP
 - Truyền không tin cậy, truyền gói tin: UDP
- ❖ Các giao thức cụ thể:
 - HTTP
 - FTP
 - SMTP, POP, IMAP
 - DNS
 - P2P: BitTorrent
- ❖ Video streaming, CDNs
- ❖ Lập trình socket: TCP socket, UDP socket

Tăng ứng dụng 2-114

Chương 2: Tổng kết

Quan trọng hơn: được học về các giao thức!

- ❖ Trao đổi giữa các thông điệp yêu cầu/đáp ứng:
 - client yêu cầu thông tin hoặc dịch vụ
 - server đáp ứng với dữ liệu, hoặc mã trạng thái
 - ❖ Định dạng thông điệp:
 - Phần tiêu đề (header): các trường với thông tin về dữ liệu
 - Dữ liệu: thông tin được truyền thông
- Các vấn đề quan trọng:**
- ❖ Tập trung và không tập trung
 - ❖ Không trạng thái và có trạng thái
 - ❖ Khả năng mở rộng
 - ❖ Truyền thông điệp tin cậy và truyền thông điệp không tin cậy
 - ❖ “Sự phức tạp tại phần cạnh của mạng”

Tăng ứng dụng 2-115

Tham khảo

- Jim Kurose, Keith Ross, “Computer Networking: A Top-Down Approach” 8th edition, Pearson, 2020.

Tăng ứng dụng 1-116